NED University of Engineering and Technology, Karachi

Computer Science and Information Technology

COMPLEX COMPUTING PROBLEM PROJECT SUBMISSION

Knowledge Based Systems (CT-359)

Robotic Arm Fault Diagnosis and Maintenance Decision Support

Project title	Submission date
Intelligent Knowledge Based System for Industrial Equipment Fault Diagnosis and Maintenance Decision Support	23 October 2026

Student Details

Sr. No.	Student Name	Roll No.	Signature
1	Saad Ali	24221	_____
2			
3			
4			
5			

This report describes the implemented, offline-capable web prototype and its synthetic evaluation. The robot type, alarm mapping, costs and historical failure data were not specified, so all non-manufacturer rules remain explicit educational assumptions. No operational accuracy or safety certification is claimed.

1. Project Instructions

The deliverable includes source code, a JSON knowledge base, synthetic cases, tests, a localhost server and this report. The application is demonstrable through a browser and does not command a robot. The project title and headings follow the prescribed template. Student rows 2–5 and signatures remain blank because no group details were supplied.

2. Problem Statement

- Represent industrial equipment, components, symptoms, faults, causes, evidence and maintenance actions with an inspectable knowledge formalism (CLO 1).
- Identify stakeholders, missing requirements, constraints and competing needs before proposing a solution (CLO 2).
- Compare representation and inference options and justify the chosen method against explanation, safety, cost and reliability criteria (CLO 3).

3. Problem Description / Project Context

A manufacturing cell uses a robotic arm with joints, tool configuration, controller, position feedback, power and an optional pneumatic gripper. An operator may see a protective stop or grip loss while controller alarms, technician inspection and maintenance history provide additional but sometimes inconsistent evidence. A single symptom can arise from more than one cause. A technician needs an auditable list of fault hypotheses and model-appropriate next checks, while a production supervisor wants to reduce downtime without bypassing safety controls.

The prototype collects categorical observations and optional source claims. It merges opposing present/absent claims into a conflict, evaluates eight explicit diagnostic rules, and derives recommendations only for fully supported hypotheses. Hazard observations are escalated independently. The design prioritizes explainability and conservative treatment of missing evidence. It cannot diagnose unmodelled failures or replace the selected robot model's service manual and qualified personnel.

4. Problem Analysis

4.1 Problem Understanding

The underlying difficulty is dispersed and incomplete maintenance knowledge. The system must distinguish a verified absence from an unperformed check, allow multiple simultaneous hypotheses, show why a rule did or did not fire, and preserve evidence provenance. The prototype targets one generic arm, not all industrial equipment in the original broad brief.

4.2 Stakeholders and Their Requirements

Stakeholder	Required outcome
Operator	Record symptoms and source notes without deciding the cause.
Maintenance technician	Inspect matched/missing/conflicting evidence, possible cause and safe next action.
Safety officer	See immediate hazard escalation regardless of diagnostic-rule completion.
Production supervisor	Compare relative downtime and cost without overriding hazard priority.
Knowledge engineer / instructor	Review and edit rules, provenance and regression results.

4.3 Constraints and Conflicting Requirements

Safety conflicts with rapid restart; a protective stop must not be dismissed merely to maintain production. Lower-cost actions may conflict with long-term reliability. Data availability conflicts with the need for decisive diagnosis; the system therefore retains unknown and conflicting states. Cost and downtime are qualitative assumptions, used only as secondary sort keys within the same evidence-status and severity group.

4.4 Root Cause / Requirement Identification

Before operational deployment, the organization must identify robot model and end effector, alarm-code semantics, approved sensor thresholds, access permissions, technician sign-off criteria, field maintenance records, source timestamps and reliability, real costs and downtime, and requirements for a shared audit store. These were not supplied. The application deliberately avoids invented numeric thresholds and marks its synthetic cases and engineering assumptions.

4.5 Assumptions

- One generic robot configuration is used for the prototype; the illustrated arm is not a verified equipment model.
- Observations are categorical: Yes, No, Unknown, or Conflict. A source claim is Yes or No. Opposing claims are not silently averaged.
- Each rule uses an AND conjunction; fully matched conditions support a hypothesis, not a confirmed root cause.
- Relative cost and downtime labels are teaching examples, not quoted plant measurements.

5. Knowledge Representation and KBS Design

5.1 Selected Knowledge Representation Formalism

The knowledge base is a set of JSON frames and production rules. Each fault frame stores component, name, required symptom keys, possible cause, severity, relative cost, downtime, maintenance action and provenance. A separate symptom dictionary maps keys to human-readable observations. Evidence records link a symptom to a source category, present/absent claim, note and timestamp. This captures equipment-to-component-to-symptom-to-fault-to-action relationships explicitly.

5.2 Knowledge Base / Ontology / Rule Base

The dataset contains 19 symptom labels and 8 rules. The eight fault families are: Payload configuration mismatch, Joint mechanical degradation, Cooling-related overheating, Position feedback fault, Pneumatic grip fault, Communication connection fault, Power supply fault, Joint holding / brake fault.

Rule	Required observations	Severity
Payload configuration mismatch	protective_stop AND payload_mismatch	High
Joint mechanical degradation	joint_noise AND joint_friction AND position_drift	High
Cooling-related overheating	overtemperature AND cooling_blocked	High
Position feedback fault	encoder_alarm AND feedback_unstable	High
Pneumatic grip fault	grip_loss AND pressure_low	High
Communication connection fault	communication_alarm AND connection_damage	Medium
Power supply fault	power_alarm AND supply_unstable	High
Joint holding / brake fault	brake_alarm AND joint_droop	Critical

Example production rule:

> IF protective_stop = Yes AND payload_mismatch = Yes
>
> THEN support payload configuration mismatch; recommend qualified review of approved payload settings.

The payload and abnormal-friction concepts follow Universal Robots guidance [1–3]. The exact conjunctions, other six fault families and all cost/downtime classes are author-created illustrative rules, not manufacturer-approved mappings.

5.3 Inference Mechanism

A three-stage forward-chaining pass first derives observed:symptom facts from Yes inputs. For each rule, it evaluates every antecedent. Any No makes the hypothesis Contradicted; otherwise a Conflict makes it Conflicted; all Yes makes it Supported; a subset Yes makes it Possible; no supporting observations makes it Unassessed. Only Supported faults derive a maintenance action. All rules are considered, so concurrent faults can be returned. The rule graph is acyclic. For F rules with at most K conditions, evaluation costs O(FK), followed by O(F log F) ordering.

5.4 Conflict Resolution Strategy

When separate sources give Yes and No for a symptom, the merge produces Conflict. The diagnostic rule cannot use that symptom as Yes. The UI lists the conflict and guided mode asks for resolution. A direct manual state entry clears prior source claims for that symptom; the user must explicitly choose this override. Hazard signals with Yes or Conflict still trigger escalation. Result sorting uses status, severity, then the selected cost or downtime tie-breaker; hazards appear in a separate alert rather than being buried in a ranking.

5.5 Explanation Facility

Each hypothesis shows its rule status, coverage of antecedents, condition-by-condition Yes/No/Unknown/Conflict groups, possible cause, action when supported and provenance. The reasoning timeline records observed facts, fired rules and derived actions. Coverage is the proportion of matched conditions, not a probability of correctness. Source claims and notes are included in exported case reports.

5.6 System Architecture

Layer	Responsibilities
Presentation (index.html, style.css)	Responsive dashboard, component map, guided checks, case history and maintenance board.
Inference (engine.mjs)	Input validation, four-state rule evaluation, ranking, hazards and trace.
Evidence (evidence.mjs)	Source categories, claim merging and source-record validation.
State (state.mjs)	Knowledge-base and backup validation.
Knowledge / cases (JSON)	Editable rules and authored synthetic scenarios.
Persistence (browser localStorage)	Drafts, cases, tasks and rule edits; export/import JSON for transfer.

6. Dataset / Knowledge Sources

Source / Dataset	Type	Size	Purpose	Citation
knowledge_base.json	Authored rules	8 faults; 19 symptoms	Diagnostic vocabulary, actions, provenance	Project file
sample_cases.json	Synthetic test cases	13 scenarios	Regression and demonstration	Project file
Universal Robots protective-stop guide	Manufacturer guidance	One online article	Payload/friction concept	[1]
Universal Robots C162	Manufacturer error-code note	One code page	Incorrect payload and centre-of-gravity example	[2]
Universal Robots e-Series handbook	Manufacturer service manual	One PDF	Protective-stop troubleshooting flow	[3]

The project dataset is authored for this course. It contains no observed plant records, no sampled sensor values, and no independently labelled holdout set. Sources [1–3] support general concepts, not universal validity of the eight constructed rules.

6. Implementation

6.1 Software / Tools / Libraries

HTML, CSS, JavaScript ES modules, JSON and Python 3 standard-library HTTP server. The browser app does not need pip or npm dependencies. Optional Node.js runs automated regression tests. The local entry point is python serve.py; open http://localhost:8000. The private hosted version shares the same static client code.

6.2 Modules and Components

The diagnostic studio collects observations, source claims, policy, asset ID and notes. Guided mode selects unresolved or unassessed conditions. Case history retains a rule snapshot per investigation for reproducibility; two cases can be compared. A maintenance board tracks follow-up tasks. The rule explorer edits and validates authored conditions and actions. The evaluation lab checks expected supported faults in 13 synthetic scenarios.

6.3 Code Structure

File	Role
dist/app.mjs	UI, events, case persistence, exports and guided flow
dist/engine.mjs	Diagnosis and rule trace
dist/evidence.mjs	Claim consensus and provenance validation
dist/state.mjs	Backup and rule-schema validation
dist/knowledge_base.json	Fault frames and sources
dist/sample_cases.json	Synthetic regression examples
serve.py	Localhost static server
test*.mjs	Inference, evidence and validation tests

6.4 User Inputs and Outputs

Inputs: symptom state or separately logged source claims, case name, asset ID, notes, priority and optional rule edits. Outputs: ranked fault hypotheses, missing checks, safety alerts, matching trace, maintenance task suggestions, case comparison, text report, JSON knowledge-base export and workspace backup. Browser storage is origin-specific. It is not a shared maintenance database or a permissioned audit log.

7. Testing and Results

The browser evaluation lab ran all 13 authored synthetic scenarios against the default rules. The expected and actual sets of supported hypotheses matched in all 13. This is a regression pass on hand-authored cases, not a measured field-accuracy result. Additional Node tests checked unknown versus No, explicit contradiction, conflict, independent hazard escalation, simultaneous faults, ordering, rule trace, malformed evidence, invalid rule conditions, invalid backups and duplicate case IDs.

Scenario	Expected supported	Observed supported	Result
Payload configuration mismatch	payload	payload	Pass
Joint mechanical degradation	mechanical	mechanical	Pass
Cooling-related overheating	thermal	thermal	Pass
Position feedback fault	encoder	encoder	Pass
Pneumatic grip fault	gripper	gripper	Pass
Communication connection fault	communication	communication	Pass
Power supply fault	power	power	Pass
Joint holding / brake fault	brake	brake	Pass
Insufficient evidence	None	None	Pass
Conflicting payload reports	None	None	Pass
Negative evidence	None	None	Pass
Two simultaneous faults	payload, gripper	payload, gripper	Pass
Immediate safety escalation	None	None	Pass

Manual browser checks also verified: selecting a demo updates findings; source Yes and No merge to Conflict and block rule support; saving a case persists after refresh; guided checks advance; two cases compare; task status persists; rule editing and the in-app evaluation work. A future independent evaluation must use expert-labelled real cases and calculate per-fault precision, recall, false alarms and missed hazards.

8. Evaluation, Limitations and Conclusion

8.1 Evaluation of the Proposed Approach

Criterion	Rule-based forward chaining	Alternative / trade-off
Explainability	Every condition and derived action is	ML would need labelled cases and

	inspectable.	post-hoc explanation.
Incomplete evidence	Unknown does not count as No.	Probabilistic reasoning could rank uncertainty if calibrated data existed.
Safety	Independent hazard alert and severity-first ordering.	Still requires qualified human procedures.
Cost / availability	Secondary qualitative ordering.	Real values and constraints must be supplied by the organization.
Maintenance	JSON rules are editable and regression-tested.	Ontology or graph could scale relationships across many equipment types.

Backward chaining could efficiently investigate a single suspected fault but would require a target hypothesis. An ontology can model richer relationships yet still needs a reasoning strategy. Bayesian inference needs reliable conditional probabilities that the brief does not supply. Forward chaining best fits broad screening from entered observations while keeping a visible explanation.

8.2 Limitations

The project uses a generic arm, only eight fault hypotheses, qualitative priorities, no live telemetry, no manufacturer-specific thresholds, no role-based access and no server-side persistence. Source categories record a claimed origin but do not authenticate it or weight credibility. Conflicts remain unresolved until a user revises evidence. Regression agreement with authored scenarios cannot establish industrial accuracy or safe operation.

8.3 Future Improvements

Select a specific model and approved manuals, acquire expert-labelled incident records, map actual alarm codes and sensor limits, and validate every rule with maintenance engineers. Add timestamped evidence imports, stronger provenance, role-controlled shared storage and review logs. Evaluate missed hazard rate and per-fault precision/recall on held-out field cases before any operational use.

8.4 Conclusion

The prototype demonstrates explicit industrial-equipment knowledge representation, conservative forward-chaining inference, conflict handling and explainable maintenance suggestions for a robotic arm. It satisfies the demonstration objectives with transparent synthetic tests. Its main value is an auditable academic model; real deployment requires model-specific knowledge acquisition and independent validation.

9. References

[1] Universal Robots - Understanding Protective Stops. https://www.universal-robots.com/articles/ur/robot-care-maintenance/understanding-protective-stops/ (accessed 2026-10-23).

[2] Universal Robots - C162 Error Code. https://www.universal-robots.com/manuals/EN/HTML/SW5_19/Content/prod-err-codes/topics/CODE_162.html (accessed 2026-10-23).

[3] Universal Robots - e-Series Service Handbook, Protective Stop. https://www.universal-robots.com/manuals/EN/PDF/SW5_24/service-manual_e-series/e-Series_Service_Manual_en.pdf (accessed 2026-10-23).

[4] NED University of Engineering and Technology, Knowledge Based Systems CT-359, Complex Computing Problem brief and supplied report template, 2026.

[5] ARM / INSIGHT source package, knowledge_base.json, sample_cases.json and tests, version 2.1, 2026.

Appendix A. Viva Demonstration and Requirement Traceability

Brief requirement	Implementation evidence
Knowledge representation (CLO 1)	JSON symptom dictionary; eight component/fault/cause/action frames; editable condition lists.
Problem analysis (CLO 2)	Stakeholder needs, incomplete requirements, constraints and assumptions in section 4.
Algorithm assessment (CLO 3)	Rule-based forward chaining compared with backward chaining, graph/ontology and Bayesian methods in section 8.1.
Conflicting information	Two opposing source claims yield Conflict and prevent a supported rule.
Explanation and safety	Condition groups, trace, independent hazard alert and human-reviewed recommendations.
Dataset and results	13 synthetic cases and executable tests; no unsupported field-accuracy claim.

Suggested five-minute demonstration: load the payload scenario and expand the reasoning timeline. Save that investigation. Add an Operator Yes and Sensor No claim for the same observation in a new case to show automatic conflict; record a qualified follow-up to resolve it. Compare both cases, create a maintenance task, and run the evaluation suite. Explain that these actions demonstrate software behavior rather than validated fault-diagnosis accuracy.

Worked example: protective_stop = Yes and payload_mismatch = Yes derive observed:protective_stop and observed:payload_mismatch. The payload rule fires and derives a fault hypothesis and its qualified maintenance recommendation. Changing payload_mismatch to Unknown leaves the fault Possible; changing it to No makes the rule Contradicted; opposing source claims make it Conflicted.